GraphPulse

Real-Time Social Network Intelligence on Hadoop, Spark, Kafka & HBase

Team Guide: what we built, how it works, and what each of us does next

Course	CSBB 422 Big Data Analytics, Lab Project
Topic	No. 15: Social Network Analysis (approved)
Repository	`https://github.com/Anirudh1621/graphpulse`
Final deadline	**13 November 2026** (only on-time submissions are eligible for an A grade)
Late window	14–20 Nov (no A grade) · after 20 Nov: C grade or lower
Guide date	11 November 2026

> **Read this first**
>
> The code is complete and in the repository. The dashboard has been tested on a laptop with a *synthetic* demo dataset. **The Spark jobs and cluster scripts have not yet run on our real 3-PC cluster**; that is today's job. Expect one or two errors on the first run. Send them to the team lead, who will get them fixed and pushed.

---

1 Project status at a glance
2 The idea in plain words
3 Architecture: how data flows
4 What each technology does for us
5 The batch pipeline, job by job
6 The real-time pipeline
7 The algorithms, explained simply
8 The dashboard
9 Repository map
10 Team roles
11 Action plan: today → submission
12 Demo day & viva preparation
13 Troubleshooting

User names in screenshots (e.g. “@silverember35”) are generated aliases. The dataset only contains anonymous numeric ids.

1 Project status at a glance

Item	Status	Notes
Topic approval	Done	Topic 15 approved.
GitHub repository	Done	All code, configs, scripts and docs pushed. Currently **private**; it must be made public (or the evaluator added) before submission.
Cluster configs & install scripts	Written	Hadoop, Spark, HBase, Kafka configs for 1 master + 2 workers; one-command install per PC.
Spark batch jobs (4 jobs)	Written	Syntax-checked. **Not yet executed on a cluster.**
Spark streaming job (5 queries)	Written	Not yet executed on a cluster.
API (FastAPI) + Dashboard (React)	Tested	Tested end-to-end on a laptop with the synthetic offline dataset. All 8 pages render.
Documentation	Done	README, cluster setup guide, command reference, demo script with viva Q&A.
3-PC cluster set up	Today	Ubuntu + install script on each PC (see Section 11).
First real pipeline run	Today / Tomorrow	Biggest remaining risk. Small dataset first, then full Twitter data.
Demo rehearsal + viva prep	Tomorrow	Everyone must be able to explain their part and run basic commands.
Team table in README, repo public, link submitted	By 13 Nov	Lead.

What the evaluators require (from the project brief) and how we cover it

Requirement	How GraphPulse meets it
Install & configure Hadoop, Kafka, Spark, HBase	All four, as a real distributed cluster: HDFS + YARN, Spark on YARN, Kafka with 3 brokers, HBase with 2 RegionServers.
Execute and explain basic commands	`docs/COMMANDS.md`: commands per tool with explanations.
Build a useful working system combining them	Batch + real-time social network analytics with a live dashboard.
1 master PC + at least 2 worker PCs	master, worker1, worker2. The dashboard's Cluster page proves it live.
A UI in any framework	React dashboard with 8 pages, served by a Python FastAPI backend.
GitHub repository link	github.com/Anirudh1621/graphpulse

2 The idea in plain words

Social networks such as Twitter/X are huge graphs: **users are nodes** and **"A follows B" is an edge**. We use a real public dataset from Stanford SNAP (the **ego-Twitter** graph, 81,306 users and 1,768,149 follow relationships) and answer questions that real social platforms care about:

Question	What GraphPulse produces
Who are the most influential users?	PageRank score + an **Influence Index** (0–100) and ranking for every user.
Which groups / communities exist?	Communities detected automatically, with quality scores (modularity, density, conductance) and a community map.
How connected is the network?	Degree distribution, triangles, clustering coefficient, reciprocity, components, and "how many hops to the top influencer" (small-world effect).
Whom should a user follow next?	"People You May Know" recommendations for every user, with the reason ("because you follow …") and a measured accuracy.
Which accounts look fake/bot-like?	A suspicion score (0–100) with reasons, from machine learning (Spark MLlib).
What is happening *right now*?	Live event stream: accounts suddenly gaining followers ("rising stars") and follow-spam bot attacks detected within seconds.
How far apart are two people?	"Six degrees of separation" path finder between any two users.

Why this is more than a basic college project

- It uses a **Lambda architecture** (a batch layer plus a real-time layer feeding one serving layer), which is the design used at companies like Twitter and LinkedIn.
- It runs on a **truly distributed cluster**: data is replicated on 2 machines, the computation runs on 2 workers, Kafka has 3 brokers, and HBase has 2 RegionServers.
- Results are **measured**. The recommender is evaluated against a baseline, there is a scaling benchmark (1 vs 2 workers), and communities are scored by modularity.
- The demo is **interactive**: we press a button to launch a simulated bot attack, and the evaluator watches Spark Streaming catch it live.

3 Architecture: how data flows

There are three layers. Read each lane left to right.

① BATCH LAYER: ACCURATE ANALYSIS OF THE WHOLE GRAPH (RUNS ON DEMAND, TAKES MINUTES)

SNAP dataset text file, 1.77M edges → **HDFS** stored as blocks on worker1 + worker2 → **Job 01** clean, de-duplicate → Parquet → **Job 02** GraphFrames + MLlib analytics → **Job 03** recommendations + evaluation → **Job 04** publish results to HBase

② SPEED LAYER: REAL-TIME EVENTS (ALWAYS RUNNING, REACTS IN SECONDS)

Event producer or dashboard buttons → **Kafka** topic `gp-events`, 3 brokers → **Spark Structured Streaming** 5 queries, time windows → **Results** Kafka `gp-insights`, HBase, HDFS data lake

③ SERVING LAYER: WHAT THE USER SEES (MILLISECONDS)

HBase per-user rows, alerts, summary documents → **FastAPI** REST API + WebSocket for live data → **React dashboard** http://master:8000

> **How the two layers meet (the "Lambda merge")**
>
> The streaming job also archives every raw event into HDFS (the data lake). The next batch run, started with `run_pipeline.sh --include-stream`, folds those new follows/unfollows back into the main graph and recomputes everything accurately. In between batch runs, live counters in HBase keep the dashboard current. For example, a profile shows "Followers (live): 101, +5 since last batch run".

Which machine runs what

PC	Hostname	Services ("daemons")
PC 1	**master**	HDFS NameNode, YARN ResourceManager, HBase Master + ZooKeeper + Thrift server, Kafka broker 1 (controller), Spark History Server, our API + dashboard. Spark jobs are *submitted* from here.
PC 2	**worker1**	HDFS DataNode (stores data), YARN NodeManager (runs Spark executors = the actual computation), HBase RegionServer, Kafka broker 2
PC 3	**worker2**	Same as worker1 (DataNode, NodeManager, RegionServer, Kafka broker 3)

4 What each technology does for us

Technology	What it is	Its job in GraphPulse
HDFS (Hadoop)	Distributed file system. Big files are split into blocks and copied to several machines.	Stores the raw dataset, cleaned Parquet tables, analysis results and the streaming data lake. Replication factor 2 means every block lives on both workers.
YARN (Hadoop)	Cluster resource manager. It hands out CPU and memory "containers" on the workers.	Runs our Spark jobs: the executors that do the work are launched on worker1 and worker2.
Spark	Fast distributed processing engine (in-memory, DataFrames, SQL).	All batch processing: cleaning, joins, aggregations, ranking.
GraphFrames	Graph library for Spark (graphs as DataFrames).	PageRank, community detection, connected components, triangle count, shortest paths, **motif finding**.
Spark MLlib	Spark's machine learning library.	KMeans clustering of user behaviour to find unusual (bot-like) accounts.
Kafka	Distributed message queue / event log. Producers write events, consumers read them.	Carries live events (follow, unfollow, like, mention) in topic `gp-events`, and Spark's results in `gp-insights`. 3 brokers, 3 partitions, each replicated twice. Runs in KRaft mode (no ZooKeeper needed).
Spark Structured Streaming	Spark for continuous data, processed in small micro-batches.	Time-window analytics on the live Kafka events: throughput, rising stars, bot-burst alerts, live counters.
HBase	Distributed NoSQL database on top of HDFS. Fast lookup of a row by its key.	The serving layer: one row per user (metrics, followers list, recommendations), alerts, dashboard documents. Answers in milliseconds.
FastAPI (Python)	Web API framework.	Reads HBase and Kafka and gives the dashboard a REST API plus a live WebSocket feed. API docs at `/docs`.
React	Frontend JavaScript framework.	The dashboard UI: interactive graphs (force-directed layout) and charts.

> **Why not just use one tool?**
> HDFS is great for scanning huge files but slow for fetching one user. HBase is the opposite. Spark batch is accurate but takes minutes; streaming is instant but only sees recent data. Combining them gives both, and that is the point of the Lambda architecture.

5 The batch pipeline, job by job

All four jobs run in order with one command, `./scripts/run_pipeline.sh`, on the master PC. Each job records its runtime in HBase, and these runs appear on the dashboard's Cluster page.

1. **Job 01: Ingest & curate** `jobs/job01_ingest.py`

   Reads the raw text edge list from HDFS. Removes comments, self-follows and duplicates, and converts ids to numbers. Marks each edge as **mutual** if both users follow each other. It also builds the list of all users and saves both tables as **Parquet**, a compressed columnar format that is much faster than text. With `--include-stream` it also merges follow/unfollow events captured by the streaming layer.

2. **Job 02: Graph analytics** `jobs/job02_graph_analytics.py`

   The core of the project. Using GraphFrames, it computes for every user: followers/following counts, **PageRank**, **community** (label propagation), **connected component**, **triangles** and **clustering coefficient**, **reciprocity**, and the **hop distance** to the 5 most important users. It combines these into an **Influence Index** and ranks everyone. It then runs **MLlib KMeans** on 6 behaviour features to produce a **suspicion score** with written reasons. Finally it computes community statistics, the network's **modularity**, the degree distribution and a global summary.

3. **Job 03: Recommendations** `jobs/job03_recommendations.py`

   Finds friend-of-friend candidates with the GraphFrames **motif** `(a)-[]->(b); (b)-[]->(c)`, meaning "a follows b, b follows c". It drops accounts that a already follows and scores the rest with **Adamic-Adar**. The top 10 per user are kept. With `--evaluate` it first hides 5% of real follows, predicts them, and measures accuracy against a "just recommend popular accounts" baseline.

4. **Job 04: Publish to the serving layer** `jobs/job04_publish_serving.py`

   Writes every user's row into HBase in parallel: each Spark partition opens its own connection to HBase. It also writes the communities and the pre-computed dashboard documents (summary, leaderboard, graph view, suspicious list, and so on). Optionally (`--snapshot`) it saves a JSON copy, which the dashboard can use as a backup if HBase is down during the demo.

Where results are stored

HDFS path	Contents
`/graphpulse/raw/edges`	Original dataset
`/graphpulse/curated/{edges,vertices}`	Cleaned graph (Parquet)
`/graphpulse/analytics/*`	users, communities, community_links, recommendations, evaluation, summary, degree histogram
`/graphpulse/lake/events`	Raw streaming events archived by Spark Streaming

HBase table	Row key	Contents
`gp_users`	user id	`m:` metrics · `n:` follower/following lists · `r:` recommendations · `s:` live counters. Pre-split into 5 regions so both workers share the load.
`gp_communities`	community id	Community statistics and top members

`gp_meta`	document name	summary, leaderboard, graph view, job run history …
`gp_alerts`	time + user	Real-time bot alerts (newest first, auto-deleted after 7 days)

6 The real-time pipeline

Producing events. `producer/event_producer.py` sends realistic events into Kafka. It picks real users from the dataset and follows the "rich get richer" pattern: popular accounts get more new followers. It has three modes:

- **baseline**: normal traffic (follows, likes, mentions, some unfollows).
- **viral**: many users suddenly follow one account, as if a post went viral.
- **bot**: one fake account follows hundreds of random users within seconds.

The dashboard's **Live Stream** page has buttons that trigger the viral and bot scenarios, which makes the demo interactive.

Processing events. `jobs/stream_graph_events.py` runs **five streaming queries** at the same time:

Query	What it does	Output
Q1 Data lake	Saves every raw event to HDFS (Parquet, one folder per day).	HDFS, used by the next batch run
Q2 Throughput	Counts events per type in 5-second windows.	Kafka → live chart
Q3 Rising stars	Counts new followers per account in a 2-minute window that moves every 10 s.	Kafka + HBase → "Rising stars" list
Q4 Threat radar	Counts follows *made* by each account in 30-second windows. 25 or more → alert. The alert is joined with the batch suspicion score (a **stream–static join**) to rate it high or critical.	Kafka + HBase → alerts
Q5 Live counters	Adds +1/−1 follower changes to atomic HBase counters.	HBase → "Followers (live)" on profiles

> **Two streaming terms you must be able to explain**
> **Window:** instead of counting "forever", we count within a time range, e.g. "follows in the last 30 seconds". A *sliding* window moves forward every few seconds.
> **Watermark:** how long Spark waits for late events (e.g. 30 s). After that the window is closed and its memory is freed, so memory stays bounded while slightly late data is still counted.

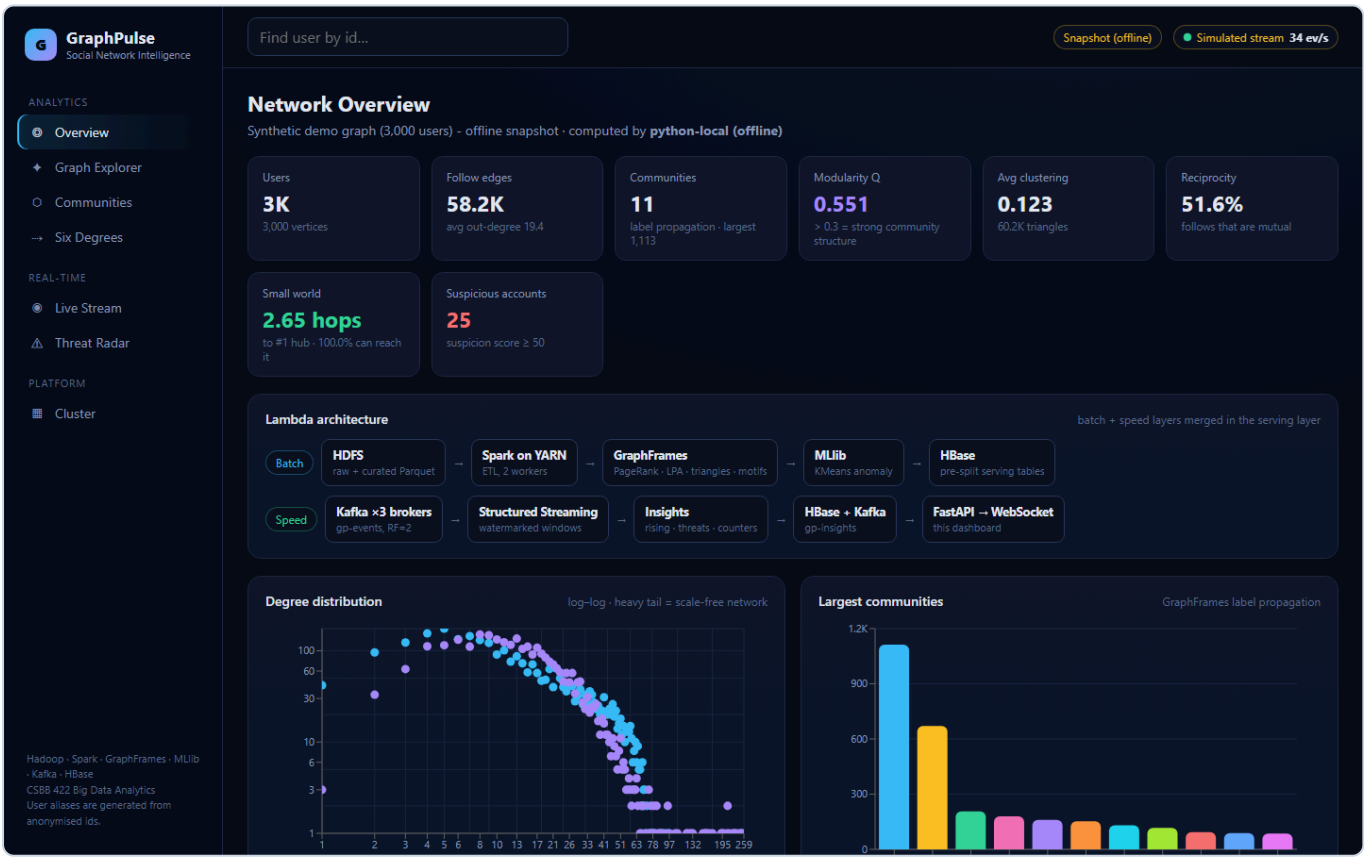
7 The algorithms, explained simply

Algorithm	Simple explanation
PageRank	You are important if *important* people follow you. Each user passes its score to the accounts it follows, split equally. This repeats about 15 times until the scores settle. The 0.15 "reset probability" means a random surfer sometimes jumps to a random user, which guarantees the scores settle. This is the algorithm Google originally used for web pages.
Influence Index	Our own 0–100 score: 65% PageRank percentile + 35% follower-count percentile.
Label Propagation (communities)	Every user starts with its own label. Each round, every user adopts the label most common among its neighbours. Tightly connected groups end up sharing one label, and each label is a community. It is fast but slightly random.
Modularity (Q)	Measures how good the community split is: more edges inside communities than random chance would give. Q > 0.3 means strong community structure.
Density / Conductance	Density = how many of the possible links inside a community exist. Conductance = share of a community's links that leave it (lower = better separated).
Triangles & clustering coefficient	A triangle is three users all connected. Clustering coefficient = what fraction of a user's friend pairs also know each other (0 to 1). Real friend groups score high; bots score near 0.
Connected components	Groups of users that can reach each other at all. In social networks, almost everyone is in one giant component.
Shortest paths (landmarks)	For every user: how many follow-hops it takes to reach each of the top-5 influencers. This shows the small-world effect (usually only 2–4 hops).
Motif finding	Searching for a pattern in the graph. `(a)→(b)→(c)` finds "friends of friends", which become recommendation candidates.
Adamic-Adar	Scores a candidate by its shared contacts, but each shared contact counts 1/log(its degree). A shared friend with 20 contacts is strong evidence; a shared celebrity with 1 million followers is weak evidence.
Recommender evaluation	Hide 5% of real follows, run the recommender, and check how many hidden follows appear in the top-10 (precision@10, recall@10, hit rate). Compare with simply recommending the most popular accounts. On the demo data ours is about 9× better; the real number will come from the cluster run.
KMeans anomaly detection	Each user becomes 6 numbers (following, followers, follow ratio, reciprocity, clustering, triangles). These are standardised and grouped into 8 behaviour clusters with KMeans. Users far from their cluster's centre are unusual. Unusual users who also follow far more accounts than follow them back get a high **suspicion score**. Simple rules add readable reasons such as "almost nobody follows it back".
Bidirectional BFS (six degrees)	To find the shortest chain between A and B, we search outward from both ends at once and stop when the searches meet. This is far less work than searching from one end. It runs live, reading follower lists from HBase.

8 The dashboard

Open **http://master:8000** once the API is running. The screenshots below were taken in offline mode with the synthetic demo dataset (3,000 users). On the cluster the same pages show the real Twitter results.

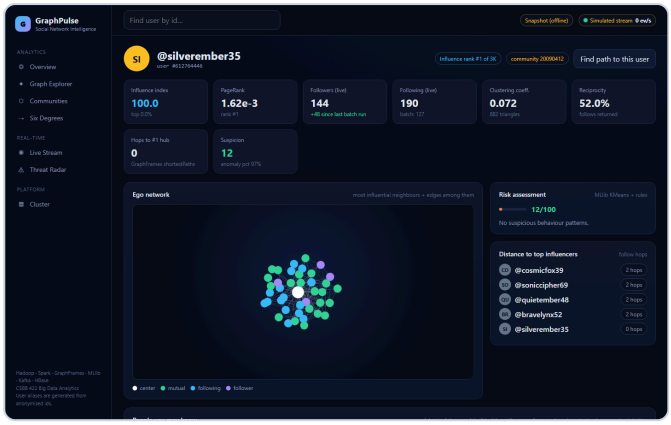
Page	What it shows
Overview	Key numbers, architecture strip, degree distribution, largest communities, top influencers, recommender accuracy, algorithm runtimes.
Graph Explorer	Top 400 users as an interactive network (size = PageRank, colour = community or suspicion). Click a node for details.
User profile	All metrics for one user, live follower count, ego network, risk assessment, distance to top influencers, “People you may know”.
Communities	Community map (bubbles + links between communities) and a table of quality metrics.
Six Degrees	Shortest follow-chain between two users, with search statistics.
Live Stream	Scenario buttons, events/second, Spark windowed throughput, rising stars, alerts, event feed.
Threat Radar	Behaviour map (bots stand out bottom-right), real-time alerts, highest-risk accounts with reasons.
Cluster	Live status of every service on all 3 PCs, HDFS/YARN usage, Kafka topics, job history and the 1-vs-2-worker benchmark.



Overview page (offline demo data).



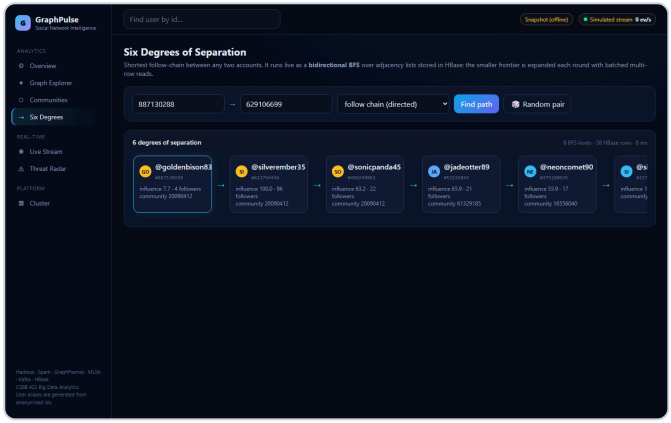
Graph Explorer: colours are communities, size is PageRank.



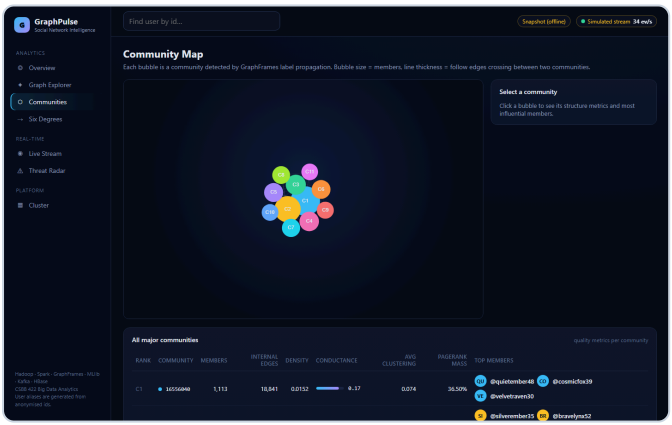
User profile with ego network and live follower delta.



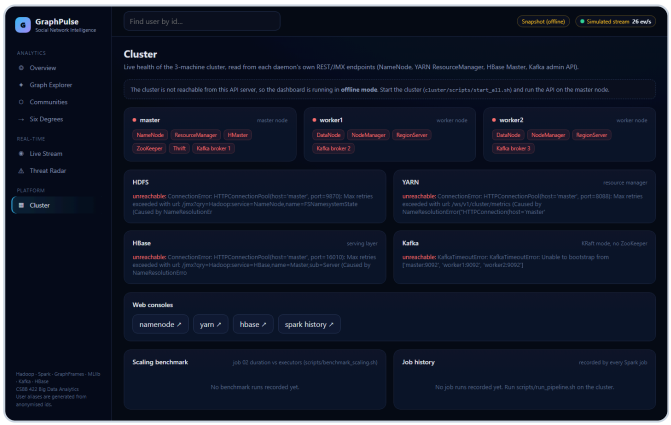
Threat Radar: bots (red) sit bottom-right; a live bot-attack alert is on the right.



Six Degrees: a 3-hop follow chain found by bidirectional BFS.



Community map and community quality table.



Cluster page. Offline it shows red “unreachable”; on the cluster every service turns green.

9 Repository map

Path	What is inside
`README.md`	Project overview (shown on the GitHub page). The team table at the bottom must be filled in.
`docs/CLUSTER_SETUP.md`	Detailed step-by-step cluster installation + troubleshooting
`docs/COMMANDS.md`	Basic commands for HDFS, YARN, Spark, Kafka, HBase with explanations (for the viva)
`docs/DEMO_SCRIPT.md`	Minute-by-minute demo plan + likely viva questions with answers
`cluster/config/`	Configuration files for Hadoop, Spark, HBase, Kafka
`cluster/scripts/`	`install_node.sh` (per PC), `setup_ssh.sh`, `bootstrap.sh` (first time), `start_all.sh`, `stop_all.sh`, `status.sh`
`cluster/hbase/create_tables.hbase`	HBase table definitions
`jobs/`	The 4 Spark batch jobs + the streaming job (+ shared helpers in `common.py`)
`producer/`	Kafka event generator (baseline / viral / bot)
`scripts/`	`download_data.sh`, `run_pipeline.sh`, `run_stream.sh`, `run_api.sh`, `benchmark_scaling.sh`
`api/`	FastAPI backend (HBase access, live Kafka bridge, path search, cluster monitor)
`ui/`	React dashboard source
`tools/make_demo_snapshot.py`	Generates the synthetic offline dataset
`demo-snapshot/`	Ready-made synthetic dataset for offline/backup mode

Software versions (installed automatically by the script)

Ubuntu 22.04 LTS · Java 11 · Python 3.10 · Hadoop 3.3.6 · Spark 3.5.1 · GraphFrames 0.8.3 · HBase 2.5.8 · Kafka 3.7.0

10 Team roles

Everyone helps with setup today. For the demo and viva, each member **owns** one area: they explain it, run its commands and answer its questions. With only three members, Member 4's area is split between Members 2 and 3.

Member	PC	Owns (explains at the demo)	Must study
Member 1 Lead (Anirudh)	master	Overall architecture, cluster bootstrap, running the pipeline and API, GitHub. Presents the overview and the live bot-attack demo.	Sections 2, 3, 6; DEMO_SCRIPT.md
Member 2	worker1	**HDFS + YARN + batch analytics**: replication, blocks, containers; Job 01 & 02 (PageRank, communities, triangles, modularity).	Sections 4, 5, 7; COMMANDS.md (HDFS, YARN, Spark)
Member 3	worker2	**Kafka + Spark Streaming**: topics, partitions, replication, brokers; the 5 streaming queries, windows, watermarks.	Sections 4, 6; COMMANDS.md (Kafka)
Member 4	(helps on any PC)	**HBase + recommendations + dashboard**: tables, row keys, pre-splitting; Job 03 & 04, Adamic-Adar, evaluation; dashboard tour.	Sections 5, 7, 8; COMMANDS.md (HBase)

> **Rule for everyone**
> Evaluators often ask *any* member about *any* part. Read Sections 2–7 of this guide completely, and be able to run at least `hdfs dfsadmin -report`, `yarn node -list`, `kafka-topics.sh --list` and `hbase shell → list`.

11 Action plan: today → submission

Before you start: what each PC needs

☐ Ubuntu **22.04 LTS** (64-bit). Install it natively/dual-boot, or as a VM (VirtualBox) with the network set to **Bridged Adapter**, at least 8 GB RAM for the VM, 4 CPUs and 40 GB disk.

☐ All 3 PCs on the **same network** (same Wi-Fi router, hotspot or LAN switch). College Wi-Fi often blocks device-to-device traffic, so a personal router or phone hotspot is safer.

☐ Internet on every PC during installation (the script downloads about 1.2 GB on each PC).

☐ Charger, and power settings set to **never sleep**.

Today (11 Nov): build the cluster

1 **Get access to the repository** Teammates

The lead shares the link `https://github.com/Anirudh1621/graphpulse`. If it is still private, send the lead your GitHub username so you can be added as a collaborator.

2 **Prepare each PC** Everyone, on their own PC

```
# 1. create the common user (same name on all 3 PCs)
sudo adduser hadoop
sudo usermod -aG sudo hadoop
su - hadoop

# 2. set the hostname: master / worker1 / worker2 (one per PC)
sudo hostnamectl set-hostname worker1

# 3. find this PC's IP address and send it to the team group
ip -4 addr | grep inet

# 4. edit /etc/hosts: add all three IPs, delete the line "127.0.1.1 ..."
sudo nano /etc/hosts
#    192.168.x.10  master
#    192.168.x.11  worker1
#    192.168.x.12  worker2

# 5. tools + code
sudo apt-get install -y git openssh-server
git clone https://github.com/Anirudh1621/graphpulse ~/graphpulse
cd ~/graphpulse
```

3 **Install the big-data stack** Everyone, on their own PC

```
./cluster/scripts/install_node.sh worker1      # use master / worker1 / worker2 matching the PC
source /etc/profile.d/bigdata.sh
java -version                                   # should print openjdk 11
```

Takes 10–30 minutes depending on internet speed. Tip: once master has downloaded everything, copy `~/gp-downloads` to the workers with a pen drive or `scp` to save time.

4 Connect the PCs & start the cluster Lead, on master

```
./cluster/scripts/setup_ssh.sh      # asks each PC's 'hadoop' password once
./cluster/scripts/bootstrap.sh      # formats storage, starts all, creates topics & tables
```

At the end a status table appears. **Every line must say UP.** Check in a browser: NameNode `http://master:9870` (2 live DataNodes) · YARN `http://master:8088` (2 nodes) · HBase `http://master:16010` (2 RegionServers).

5 First test run with the small dataset Lead, on master

```
./scripts/download_data.sh --small     # Facebook graph, 4k users: quick test
./scripts/run_pipeline.sh
```

If anything fails, copy the **full error text** (the last ~40 lines of the terminal) and send it to the lead / AI assistant. After a fix is pushed, run `git pull` on **all three** PCs.

Tomorrow (12 Nov): real data, live demo, rehearsal

6 Full Twitter dataset Lead

```
hdfs dfs -rm -r /graphpulse/raw/edges/facebook_combined.txt
./scripts/download_data.sh             # 81k users, 1.77M edges
./scripts/run_pipeline.sh              # roughly 5–20 minutes
./scripts/benchmark_scaling.sh         # optional: 1 vs 2 workers chart
```

7 Start the live system (3 terminals on master) Lead

```
./scripts/run_stream.sh                                                    # terminal 1
python3 producer/event_producer.py --edges data/twitter_combined.txt --rate 30   # terminal 2
./scripts/run_api.sh                                                       # terminal 3
```

Open **http://master:8000** from any PC. Go to Live Stream and press “Launch bot attack”; an alert should appear within about 15 seconds.

8 Rehearse Everyone

Do one complete run-through using `docs/DEMO_SCRIPT.md`, about 12 minutes. Each member presents their own area and runs their commands from `docs/COMMANDS.md`. Then quiz each other with the viva questions at the end of the demo script.

By 13 Nov: submit

9 Finalise and submit Lead

- Fill in the team table (names, roll numbers, responsibility) at the bottom of `README.md`, commit and push.
- Make the repository **public**: GitHub → Settings → Danger Zone → Change visibility.
- Send the repository link with the group name, roll numbers and approved title through the channel your instructor uses. The topic approval went to `priten@nitdelhi.ac.in`.

12 Demo day & viva preparation

Setup checklist (30 minutes before)

☐ All 3 PCs on, on the same network, `/etc/hosts` still correct (IP addresses can change after reconnecting!).

☐ `./cluster/scripts/start_all.sh` on master → everything UP.

☐ Streaming job, producer and API running; dashboard open in the browser.

☐ Browser tabs open: dashboard, NameNode (9870), YARN (8088), HBase (16010).

☐ Backup: a laptop with the repo and the JSON snapshot copied from master (`~/graphpulse-snapshot`). Run it with `GP_BACKEND=snapshot` if the cluster fails, and say honestly what failed.

Demo order (about 12 minutes)

Min	Show	Presenter
0–1	Overview page: problem, dataset, Lambda architecture	Member 1
1–3	Cluster page + `status.sh`, `hdfs dfsadmin -report`, `hdfs fsck … -locations`, YARN UI	Member 2
3–5	Overview KPIs, degree distribution, Graph Explorer, Communities (PageRank, LPA, modularity)	Member 2
5–7	User profile, recommendations + evaluation card, Six Degrees, HBase shell `get`	Member 4
7–10	Live Stream: viral surge → rising star; bot attack → critical alert; Kafka console consumer	Member 3 / 1
10–12	Live follower counter, Lambda merge, scaling benchmark, GitHub repo	Member 1

Top questions to prepare (full answers in docs/DEMO_SCRIPT.md)

1. Why GraphFrames instead of GraphX? How does PageRank / label propagation work?
2. What is the clustering coefficient? What does modularity tell us?
3. How does the recommender work, and how did you evaluate it?
4. How is the suspicion score computed? Why KMeans?
5. What is a window and a watermark in Spark Streaming?
6. What is the Lambda architecture? How do batch and streaming results combine?
7. Why HBase and not MySQL or plain HDFS? Why pre-split the table?
8. Why 3 Kafka brokers? What are partitions and the replication factor?
9. What happens if a worker PC is switched off during the demo?
10. How would the system handle 100× more data?

13 Troubleshooting

Problem	Fix
NameNode shows 0 live DataNodes	`/etc/hosts` wrong on some PC, or the `127.0.1.1` line still present. Firewall: `sudo ufw disable` on all PCs.
PCs cannot ping each other	College Wi-Fi "client isolation". Use a personal router or phone hotspot; VMs must use **Bridged** networking.
Script error `$'\r': command not found`	`sed -i 's/\r$//' cluster/scripts/*.sh scripts/*.sh`
Spark job stays in ACCEPTED	Not enough memory on the workers. Lower `spark.executor.memory` in `cluster/config/spark/spark-defaults.conf`, re-run the install script's copy step or copy the file to `/opt/bigdata/spark/conf/`.
`No module named graphframes`	master needs internet for the first run (downloads the package).
HBase RegionServer stops	Clocks out of sync: `sudo timedatectl set-ntp true` on all PCs, then restart.
Kafka broker does not start	`rm -rf /opt/bigdata/data/kafka/*` on all PCs and re-run the Kafka format step of `bootstrap.sh`.
Dashboard shows "Snapshot (offline)"	API cannot reach HBase: start the Thrift server with `hbase-daemon.sh start thrift -p 9090`, then restart the API.
Anything else	Copy the full error text and the command you ran, and send them to the lead. Don't change configs randomly; that makes problems harder to trace.

> **Daily routine once the cluster works**
> Start: `./cluster/scripts/start_all.sh` → check `status.sh`. Stop (before shutting down the PCs): `./cluster/scripts/stop_all.sh`. Get updates: `git pull` on all PCs.